

Sistema de Gestión de Pedidos con API Gateway, Lambda, y DynamoDB

Empresa: OrderFlow

Contexto: OrderFlow es una startup dedicada a la entrega de pedidos a domicilio. La empresa está en plena expansión y busca una arquitectura escalable que permita gestionar pedidos sin grandes costos de infraestructura, al tiempo que mantenga un flujo continuo de despliegues para mejorar el tiempo de entrega de nuevas funcionalidades.

Necesidad: La empresa necesita una solución que permita a los clientes hacer pedidos a través de una aplicación web. Estos pedidos deben ser procesados y almacenados en una base de datos, y el equipo de logística debe tener acceso a los pedidos para su rápida entrega.

Objetivo del proyecto: Desarrollar una aplicación web donde los clientes puedan realizar pedidos y ver el estado de su pedido. La solución debe ser desplegada utilizando un pipeline CI/CD para garantizar que las actualizaciones sean automáticas y sin interrupciones. Los pedidos deben almacenarse en DynamoDB y la aplicación debe escalar según la demanda.

Descripción del Proyecto

Este proyecto requiere el desarrollo de una aplicación web que permita a los clientes hacer pedidos y visualizarlos en tiempo real. La aplicación se basará en una arquitectura serverless, con API Gateway y AWS Lambda para el backend y DynamoDB para el almacenamiento de datos. Además, debe contar con un pipeline CI/CD que despliegue automáticamente las nuevas versiones de la aplicación.

Requisitos del Proyecto

Backend:

AWS Lambda: Desarrollar funciones Lambda que gestionen el flujo de pedidos, desde la creación hasta el cambio de estado (e.g., "en preparación", "en camino", "entregado").

API Gateway: Configurar una API con API Gateway que exponga los endpoints para crear y consultar el estado de los pedidos.

DynamoDB: Crear una tabla DynamoDB para almacenar los detalles de los pedidos y su estado. Esta tabla debe ser diseñada para un acceso rápido y optimizado.

Frontend:

Aplicación Web: Crear una aplicación web con un formulario para realizar pedidos y visualizar su estado.

Integración con el Backend: La aplicación debe realizar llamadas a la API en API Gateway para enviar y recuperar información de los pedidos en tiempo real.

Pipeline CI/CD:

GitHub Actions o AWS CodePipeline:

Configurar un pipeline CI/CD que se active con cada cambio en la rama principal del repositorio.

Automatización del Despliegue: Asegurar que las actualizaciones del código se desplieguen automáticamente a AWS, reduciendo el tiempo de inactividad.

Notificaciones: Configurar notificaciones en caso de fallos en el pipeline para que el equipo pueda responder rápidamente.

Monitoreo y Escalabilidad:

CloudWatch: Configurar métricas y alarmas en CloudWatch para supervisar las invocaciones de Lambda, el tráfico en API Gateway y la tasa de consumo de la tabla de DynamoDB.

Criterios de Éxito

- **Funcionalidad Completa:** Los usuarios deben poder realizar pedidos y verificar su estado.
- **CI/CD Efectivo:** El pipeline CI/CD debe desplegar las actualizaciones automáticamente a Lambda, API Gateway, y DynamoDB sin problemas.
- **Escalabilidad y Monitoreo:** El sistema debe ser escalable para soportar picos de pedidos y debe estar monitorizado para asegurar la continuidad del servicio.
- **Integración:** De los servicios SQS y SNS con las funciones lambdas requeridas para el proyecto.

Entrega Final

- **Repositorio GitHub:** Los estudiantes deben entregar un repositorio que contenga el código del backend (Lambda y configuraciones de API Gateway y DynamoDB) y del frontend, junto con el pipeline CI/CD.
- **Documentación:** Incluir un README con instrucciones de cómo ejecutar y desplegar la aplicación y la arquitectura del sistema.

- **Presentación:** Preparar una presentación que cubra la arquitectura de la solución, los desafíos enfrentados y cómo se implementó el CI/CD.
- **Arquitectura:** Incluir un diagrama visual de la arquitectura mostrando cómo interactúan los componentes del sistema.